



PANDAS FOR DATA SCIENCE & AI: MASTERCLASS NOTES

Course Objective: Master high-performance data manipulation using Vectorized Operations. Move away from standard loops and learn to handle massive datasets for Machine Learning pipelines.



SESSION 1: Core Architectures & Data Input

Pandas operates primarily on two structural objects:

1. **Series:** A 1-Dimensional labeled array (akin to a single column).
2. **DataFrame:** A 2-Dimensional labeled data

structure (akin to an Excel spreadsheet or SQL table).

Complete Implementation Code

```
import pandas as pd  
import numpy as np
```

#1. Instantiating a DataFrame from scratch

```
developer_data = {  
    'Name': ['Aman', 'Siddharth', 'Riya', 'Vikram'],  
    'Role': ['ML Engineer', 'Data Scientist', 'Python  
Dev', 'AI Researcher'],  
    'Experience_Yrs': [4, np.nan, 2, 5],  
    'Base_Salary': [95000, 105000, 72000, 120000]  
}  
df = pd.DataFrame(developer_data)
```

#2. Reading external production data (CSV / Excel)

```
df =  
pd.read_csv('production_dataset.csv')
```

#3. Structural Inspection Methods

```
print("--- FIRST 2 RECORDS ---")  
print(df.head(2))
```

```
print("\n--- DATA TYPES & NULL COUNTS ---")  
print(df.info\(\))
```

```
print("\n--- SHAPE (ROWS, COLUMNS) ---")  
print(df.shape)
```

⚡ SESSION 2: High-Performance Data Filtering

Avoid iterating with manual for loops. Pandas uses underlying NumPy arrays to execute operations in C-speed across vector blocks.

`.loc[]`: Label-based indexing (uses actual row/column names).

`.iloc[]`: Integer-position based indexing (uses raw coordinate numbers starting at 0).

Complete Implementation Code

#1. Conditional Masking (Boolean Indexing)

#Target: Highly experienced devs making over 80k

```
senior_high_earners = df.loc[(df['Base_Salary'] >
80000) & (df['Experience_Yrs'] >= 3)]
print("--- FILTERED RESULTS ---")
print(senior_high_earners)
```

#2. Positional Slicing via .iloc

**#Target: Grab the first 3 rows and
the first 2 columns**

```
sub_matrix = df.iloc[0:3, 0:2]
print("\n--- SLICED SUB-MATRIX ---")
print(sub_matrix)
```



SESSION 3: Handling Missing Values (Data Preprocessing)

Real-world datasets contain missing markers (NaN). Machine learning models built with Scikit-Learn will throw fatal errors if empty slots are passed into them.

Complete Implementation Code

#1. Audit missing cells across columns

```
print("--- MISSING VALUES PER COLUMN ---")  
print(df.isnull().sum())
```

#2. Strategy A: Drop data (Use only when missing data

represents less than 5% of entries)

```
df_dropped = df.dropna()
```

#3. Strategy B: Imputation (Fill gaps using statistical metrics like mean or median)

```
median_experience =
```

```
df['Experience_Yrs'].median()
```

```
df['Experience_Yrs'] =
```

```
df['Experience_Yrs'].fillna(median_experience)
```

```
print("\n--- DATA AFTER MEDIAN IMPUTATION  
---")
```

```
print(df)
```



SESSION 4: Grouping & Data Aggregation

Aggregating lets you extract deep statistical trends by grouping common categorical values together.

Complete Implementation Code

#Calculate the average salary offered across different roles

```
average_salary_by_role = df.groupby('Role')  
['Base_Salary'].mean().reset_index()  
print("--- ROLE-BASED AGGREGATION ---")  
print(average_salary_by_role)
```




SHOWCASE CAPSTONE PROJECT: AUTOMATED DATA CLEANING ENGINE

This capstone project combines data ingestion, data imputation, feature engine mapping, and categorical statistical aggregation into a single executable script.

```
import pandas as pd  
import numpy as np
```

#Step 1: Simulate Incoming Corrupted Production Data

```
raw_logs = {  
'User_ID': [1001, 1002, 1003, 1004, 1005, 1006],  
'Platform': ['Android', 'iOS', 'Web', 'Android', 'iOS',
```

```
'Web'],  
'Daily_Minutes': [45, np.nan, 12, 110, np.nan, 15],  
'Purchased_Premium': [1, 0, 0, 1, 1, 0]  
}  
  
raw_df = pd.DataFrame(raw_logs)  
print("=== PHASE 1: INCOMING RAW  
UNCLEANED LOGS ===")  
print(raw_df)
```

#Step 2: Build the Data Transformation Pipeline

```
def clean_and_process_pipeline(input_df):  
    processed_df = input_df.copy()  
  
    # Impute missing minutes with the average  
    platform baseline usage  
    mean_minutes =  
    processed_df['Daily_Minutes'].mean()
```

```
processed_df['Daily_Minutes'] =  
processed_df['Daily_Minutes'].fillna(mean_minut  
es)
```

```
# Feature Engineering: Flag heavy users (greater  
than 40 minutes active)
```

```
processed_df['Is_Heavy_User'] =  
processed_df['Daily_Minutes'] > 40
```

```
return processed_df
```

#Step 3: Run the Pipeline

```
clean_df = clean_and_process_pipeline(raw_df)  
print("\n=== PHASE 2: PIPELINE OUTPUT  
CACHE ===")  
print(clean_df)
```

#Step 4: Extract Key Business Insight for AI Target Models

```
platform_metrics = clean_df.groupby('Platform')  
['Purchased_Premium'].sum().sort_values(ascending=False)  
print("\n=== PHASE 3: TOTAL CONVERSIONS BY  
OPERATING PLATFORM ===")  
print(platform_metrics)
```